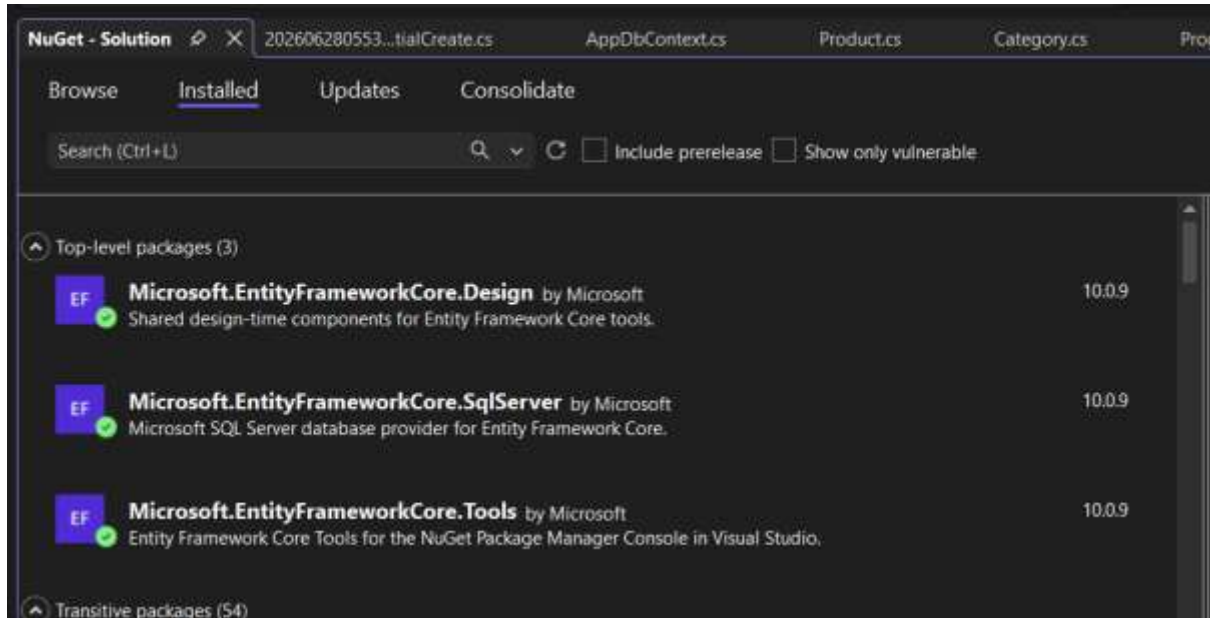


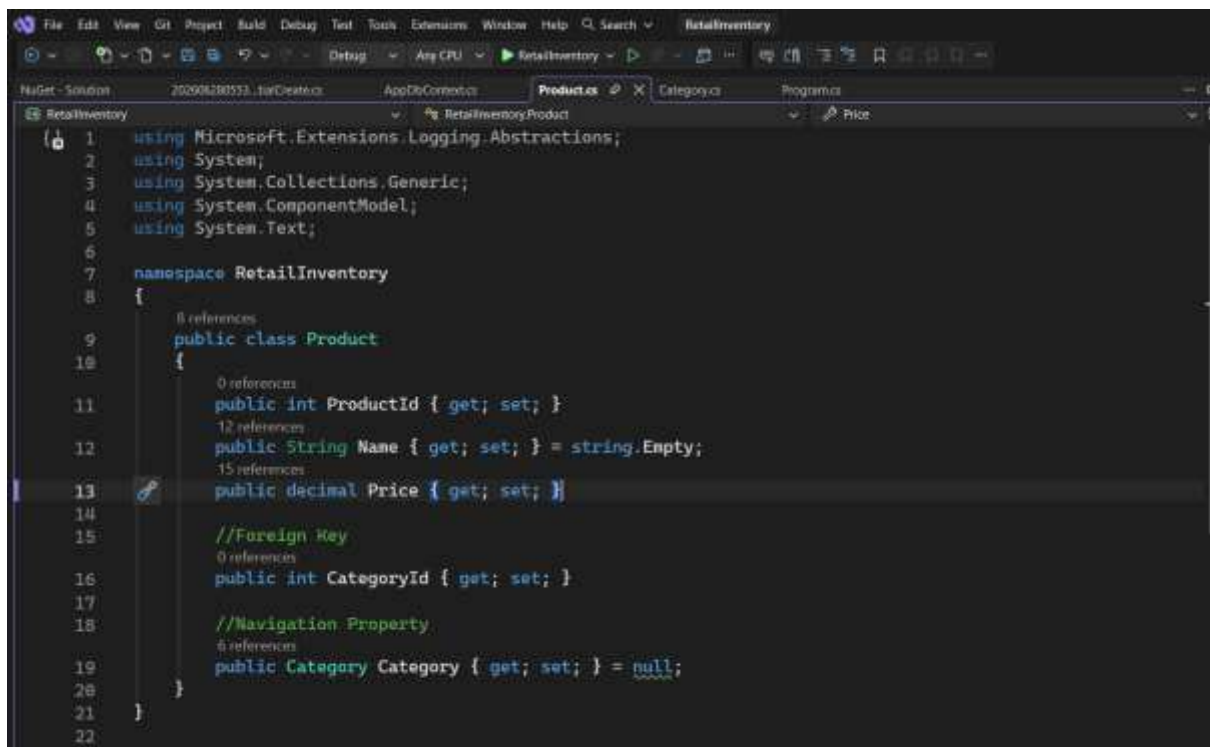
EF Core 8.0 Solved Exercise

Lab 1: Understanding ORM with a Retail Inventory System

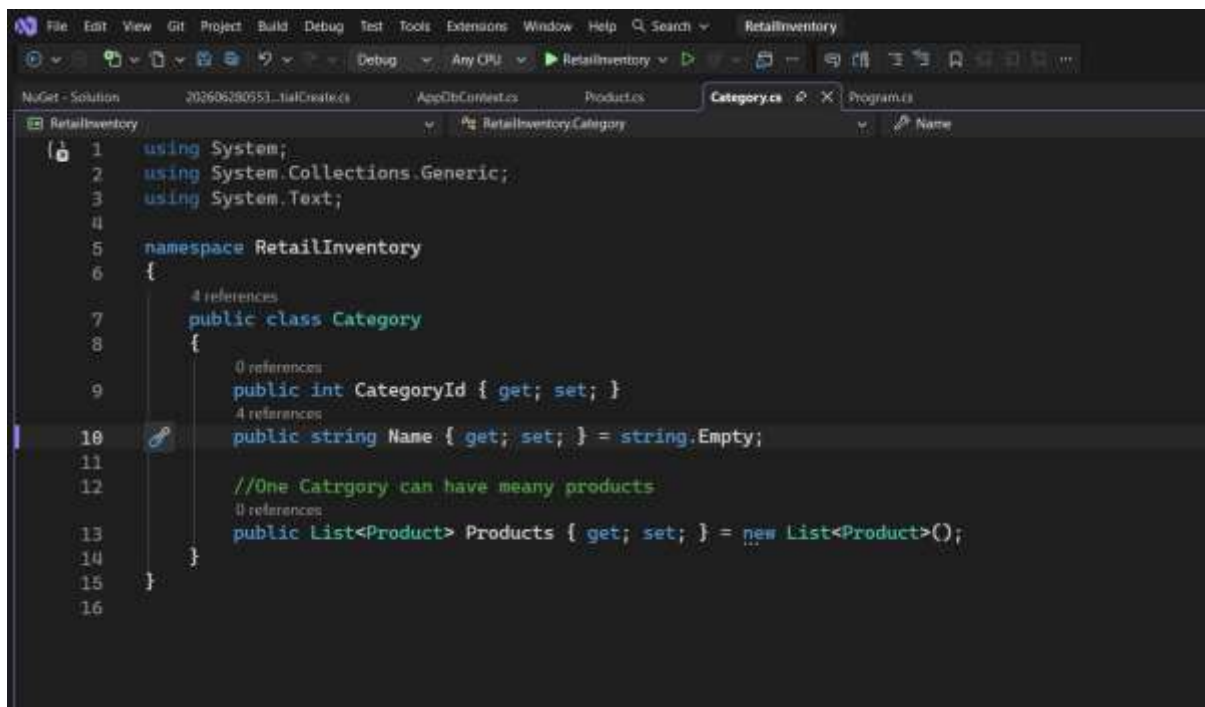


Lab 2: Setting Up the Database Context for a Retail Store

Product :



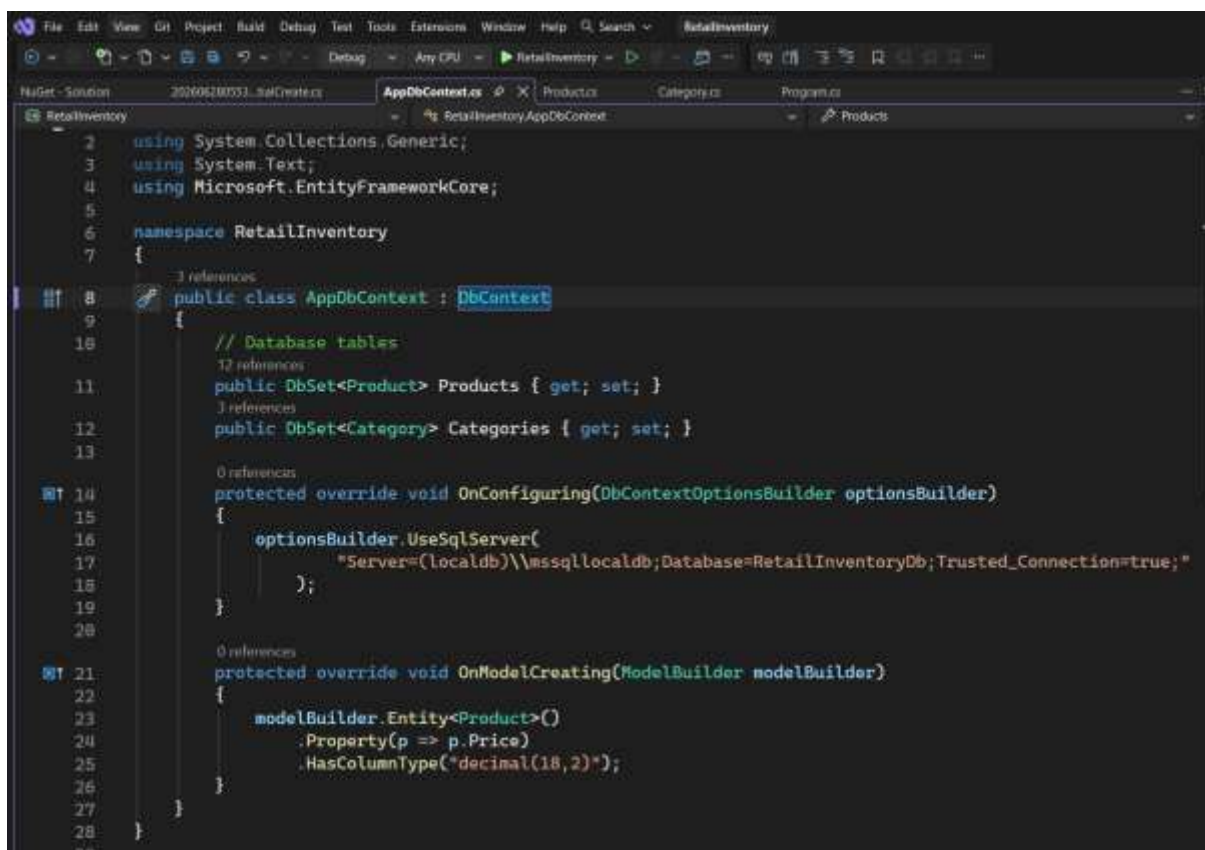
Category :



The screenshot shows the Visual Studio IDE with the 'RetailInventory' project selected. The 'Category.cs' file is open, displaying the following code:

```
1 using System;
2 using System.Collections.Generic;
3 using System.Text;
4
5 namespace RetailInventory
6 {
7     4 references
8     public class Category
9     {
10         0 references
11         public int CategoryId { get; set; }
12         4 references
13         public string Name { get; set; } = string.Empty;
14
15         //One Catrgory can have meany products
16         0 references
17         public List<Product> Products { get; set; } = new List<Product>();
18     }
19 }
```

AppDbContext :



The screenshot shows the Visual Studio IDE with the 'RetailInventory' project selected. The 'AppDbContext.cs' file is open, displaying the following code:

```
2 using System.Collections.Generic;
3 using System.Text;
4 using Microsoft.EntityFrameworkCore;
5
6 namespace RetailInventory
7 {
8     3 references
9     public class AppDbContext : DbContext
10     {
11         // Database tables
12         12 references
13         public DbSet<Product> Products { get; set; }
14         3 references
15         public DbSet<Category> Categories { get; set; }
16
17         0 references
18         protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
19         {
20             optionsBuilder.UseSqlServer(
21                 "Server=(localdb)\\mssqllocaldb;Database=RetailInventoryDb;Trusted_Connection=true;"
22             );
23         }
24
25         0 references
26         protected override void OnModelCreating(ModelBuilder modelBuilder)
27         {
28             modelBuilder.Entity<Product>()
29                 .Property(p => p.Price)
30                 .HasColumnType("decimal(18,2)");
31         }
32     }
33 }
```

Lab 3: Using EF Core CLI to Create and Apply Migrations

```
PM> Add-Migration InitialCreate
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for more information if this takes too long.
Applying migration '20268628855388_InitialCreate'.
Done.
PM> |
```

Lab 4: Inserting Initial Data into the Database

```
22 context.Categories.RemoveRange(existingCategories);
23 await context.SaveChangesAsync();
24 }
25
26 // Add Categories (only if database is empty)
27
28 Console.WriteLine("Adding Categories...\n");
29
30 //create categories
31 var electronics = new Category { Name = "Electronics" };
32 var groceries = new Category { Name = "Groceries" };
33
34 await context.Categories.AddRangeAsync(electronics, groceries);
35 await context.SaveChangesAsync();
36 Console.WriteLine("Categories added successfully!\n");
37
38 //create products linked to categories
39 var product1 = new Product { Name = "Laptop", Price = 75000, Category = electronics };
40 var product2 = new Product { Name = "Rice Bag", Price = 1200, Category = groceries };
41 var product3 = new Product { Name = "Phone", Price = 30000, Category = electronics };
42 var product4 = new Product { Name = "Wheat Bag", Price = 800, Category = groceries };
43
44 await context.Products.AddRangeAsync(product1, product2, product3, product4);
45 await context.SaveChangesAsync();
46 Console.WriteLine("Products added successfully to respective Categories!\n");
47
```

Lab 5: Retrieving Data from the Database

```
49 //Retrieving data
50 Console.WriteLine(">>> Retrieving Data\n");
51
52 Console.WriteLine("All Products : ");
53 var allProducts = await context.Products.ToListAsync();
54 foreach (var p in allProducts)
55 {
56     Console.WriteLine($"    {p.Name} : Rs.{p.Price}");
57 }
58
59
60 //Find Product By Id
61 Console.WriteLine("\n Find Product by ID: 1 : ");
62 var productById = await context.Products.FindAsync(1);
63 Console.WriteLine($"    Found : {productById?.Name}");
64
65
66 //FirstOrDefault With Condition
67 Console.WriteLine("\n First product with price > 50000 : ");
68 var expensive = await context.Products
69     .FirstOrDefaultAsync(p => p.Price > 50000);
70 Console.WriteLine($"    Expensive Product : {expensive?.Name ?? "Not Found"}");
71
```

Lab 6: Updating and Deleting Records

```
72 |
73 | // Update Price Of Laptop
74 | var laptopToUpdate = await context.Products
75 |     .FirstOrDefaultAsync(p => p.Name == "Laptop");
76 |
77 | if(laptopToUpdate != null)
78 | {
79 |     Console.WriteLine($"\\nLaptop Price BEFORE update : Rs.{laptopToUpdate.Price}");
80 |     laptopToUpdate.Price = 50000;
81 |     await context.SaveChangesAsync();
82 |     Console.WriteLine($"\\nLaptop Price AFTER update : Rs.{laptopToUpdate.Price}");
83 | }
84 |
85 |
86 | // Delete Rice Bag
87 | var toDelete = await context.Products
88 |     .FirstOrDefaultAsync(p => p.Name == "Rice Bag");
89 |
90 | if(toDelete != null)
91 | {
92 |     context.Products.Remove(toDelete);
93 |     await context.SaveChangesAsync();
94 |     Console.WriteLine("\\n'Rice Bag' Deleted Successfully");
95 | }
96 | else
97 | {
98 |     Console.WriteLine("\\n'Rice Bag' Not Found");
99 | }
```

Lab 7: Writing Queries with LINQ

```
182 | // LINQ Queries
183 | Console.WriteLine("\\n>>> LINQ Queries\\n");
184 |
185 | Console.WriteLine("Products with Price > 1000, sorted high to low:");
186 | var filtered = await context.Products
187 |     .Where(p => p.Price > 500)
188 |     .OrderByDescending(p => p.Price)
189 |     .ToListAsync();
190 |
191 | foreach (var p in filtered)
192 | {
193 |     Console.WriteLine($"    - {p.Name} : Rs.{p.Price}");
194 | }
195 |
196 | // DTO projection
197 | Console.WriteLine("\\nProduct Name & Price only (DTO projection):");
198 | var productDTO = await context.Products
199 |     .Select(p => new { p.Name, p.Price })
200 |     .ToListAsync();
201 |
202 | foreach(var p in productDTO)
203 | {
204 |     Console.WriteLine($"    - {p.Name} => Rs.{p.Price}");
205 | }
206 |
207 | // With Category Name
208 | Console.WriteLine("\\nProducts with Category Name:");
209 | var withCategory = await context.Products
210 |     .Include(p => p.Category)
211 |     .Select(p => new { p.Name, p.Price, CategoryName = p.Category.Name})
212 |     .ToListAsync();
213 |
214 | foreach(var p in withCategory)
215 | {
216 |     Console.WriteLine($"    - {p.Name} | Rs.{p.Price} | {p.CategoryName}");
217 | }
```

OutPut :

```
Microsoft Visual Studio Debug Console
=== Retail Inventory System ===

Adding Categories...

Categories added successfully!

Products added successfully to respective Categories!

>>> Retrieving Data

All Products :
  - Laptop : Rs.75000
  - Rice Bag : Rs.1200
  - Phone : Rs.30000
  - Wheat Bag : Rs.800

Find Product by ID: 1 :
  Found :

First product with price > 50000:
  Expensive Product : Laptop

Laptop Price BEFORE update : Rs.75000

Laptop Price AFTER update : Rs.50000

'Rice Bag' Deleted Successfully

>>> LINQ Queries

Products with Price > 1000, sorted high to low:
  - Laptop : Rs.50000
  - Phone : Rs.30000
  - Wheat Bag : Rs.800

Product Name & Price only (DTO projection):
  - Laptop => Rs.50000.00
  - Phone => Rs.30000.00
  - Wheat Bag => Rs.800.00

Products with Category Name:
  - Laptop | Rs.50000.00 | Electronics
  - Phone | Rs.30000.00 | Electronics
  - Wheat Bag | Rs.800.00 | Groceries
```